

9 PRIVITAK

9.1 Osnove prikaza brojeva u računalu

9.1.1 Binarni brojevni sustav

9.1.1.1 Negativni binarni brojevi

9.1.1.2 Razlomljeni binarni brojevi

9.1.2 Oktalni brojevni sustav

9.1.3 Heksadecimalni brojevni sustav

9.1.4 Prikaz realnih brojeva u računalu

9.1.4.1 Numeričke pogreške

9.2 Temeljni pojmovi programiranja

9.3 Sintaksa programskih jezika

9.4 Rječnik pojmova programskog jezika C++

9 PRIVITAK

9.1 Osnove prikaza brojeva u računalu

9.1.1 Binarni brojevni sustav

Brojevni sustav s bazom brojanja **B** ima znamenke **0, 1, 2, ... ,B-1**

Npr. u dekadskom sustavu **B=10**, a znamenke su **0, 1, 2, ..., 8 i 9**.

Ako je baza **B=2** dobiva se **binarni brojevni sustav**, čije znamenke su **0 i 1**.

Iz engleskog **B**inary digi**T** nastalo je ime za najmanju količinu informacije **BIT**.

Primjer zapisivanja brojeva:

$$57_{10} = 5 * 10^1 + 7 * 10^0 = 1*2^5 + 1*2^4 + 1*2^3 + 0*2^2 + 0*2^1 + 1*2^0 = 1\ 1\ 1\ 0\ 0\ 1_2$$

Za binarno prikazivanje informacija je potreban najveći broj elemenata u usporedbi s ostalim prikazima, a broj bita za prikaz brojeva je iz tehničkih razloga ograničen .

Uređaji koji obrađuju i pohranjuju binarne informacije fizički se izvode pomoću elektroničkih elementa s 2 stabilna stanja (sklopka), koji su veoma brzi i jeftini.

Primjer pretvorbe dekadskog broja u binarni:

Binarni broj tvore ostaci dijeljenja s 2, odozdo prema gore:

57 : 2 = 28	1	1	1	0	0	1
1						
28 : 2 = 14						
0						
14 : 2 = 7						
0						
7 : 2 = 3						
1						
3 : 2 = 1						
1						
1 : 2 = 0						
1						

9.1.1.1 Negativni binarni brojevi

Registar je skup memorijskih elemenata koji pamte znamenke binarnog broja što znači da je za određeno računalo unaprijed propisana dužina registra, a time i brojevno područje unutar kojeg se kreću brojevi.

Primjer: Operacija $7 - 5$ u računalu s registrom od 4 bita obaviti će se kao $7 + (-5)$. Binarni prikaz broja -5 je sljedeći:

Pozitivni broj **0 1 0 1**

Komplement do baze-1 **1 1 1 1**
(jedinični komplement) **- 0 1 0 1**
 1 0 1 0

Komplement do baze **1 0 1 0**
(dvojni komplement) **+ 0 0 0 1**
 1 0 1 1

Dokaz da je dobiveni broj -5

Operacija oduzimanja $7 - 5$

1 0 1 1 (-5)
+ **0 1 0 1** $(+5)$
 0 0 0 0

Preliv 1

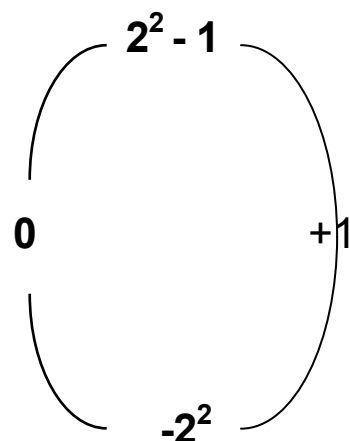
0 1 1 1 (7)
+ **1 0 1 1** (-5)
 0 0 1 0

Preliv 1

U registru s 3 bita, ako je prvi bit predznak mogu se prikazati sljedeći brojevi:

Dekadski broj Binarni broj

0	000
1	001
2	010
3	011
-4	100
-3	101
-2	110
-1	111



Za $n=3$ dobije se interval $[-2^2, 2^2 - 1]$, općenito $[-2^{n-1}, 2^{n-1} - 1]$.

Za $n=8$ taj je interval $[-2^7, 2^7-1]$, tj. $[-128, 127]$.

9.1.1.2 Razlomljeni binarni brojevi

Razlomljeni binarni brojevi sadrže "binarnu točku", analogno decimalnom zarezu, odnosno točki u anglo-američkoj notaciji.

Primjer prikaza razlomljenih brojeva:

$$\begin{aligned} 5.75_{10} &= 5 * 10^0 + 7 * 10^{-1} + 5 * 10^{-2} = \\ &= 1*2^2 + 0*2^1 + 1*2^0 + 1*2^{-1} + 1*2^{-2} = 1\ 0\ 1.\ 1\ 1_2 \end{aligned}$$

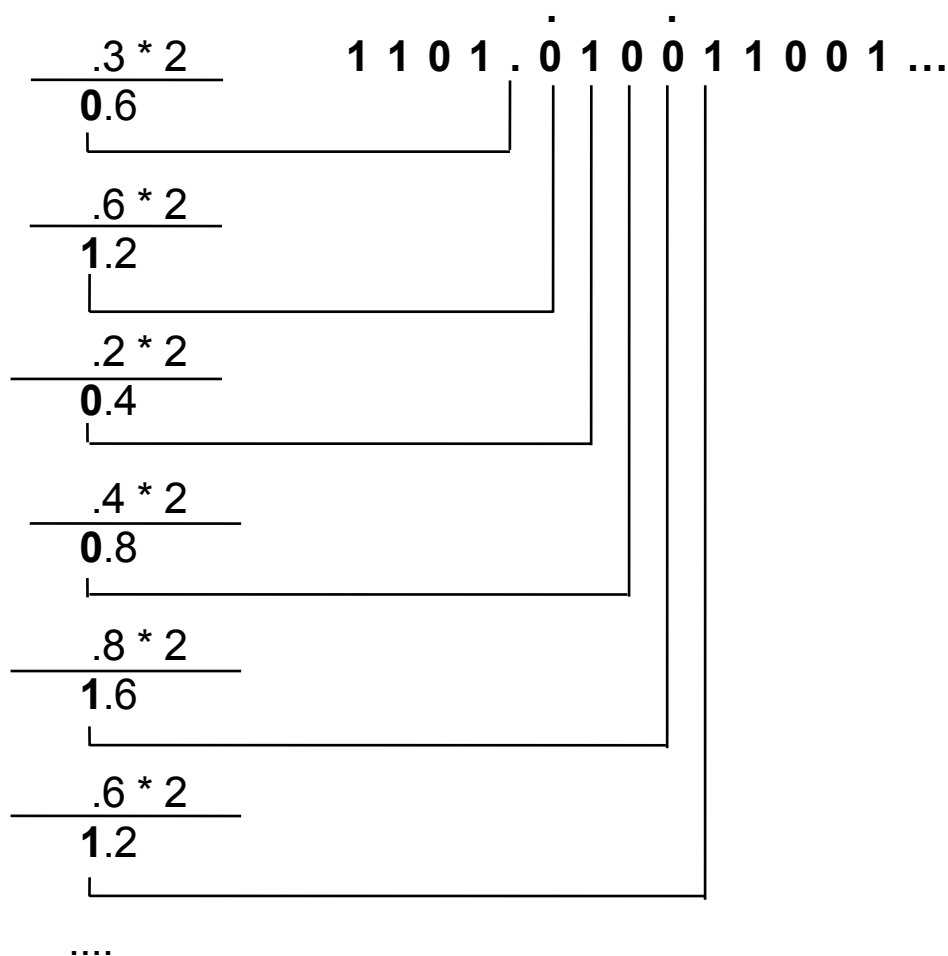
Primjer pretvaranja decimalnog broja u binarni:

Cjelobrojni dio dekadskog broja pretvara se u binarni uzastopnim dijeljenjem, a decimalni uzastopnim množenjem s 2, gdje cjelobrojni dio dobivenih produkata tvori znamenke binarnog razlomka.

$$\begin{array}{rcl} 1.25 & = & 1 + .25 \\ & & \swarrow \\ .25 * 2 & & 1\ .\ 0\ 1 \\ \hline 0.50 & & \\ & & \swarrow \\ .5 * 2 & & \\ \hline 1.0 & & \end{array}$$

Primjer pretvaranja decimalnih brojeva koji se ne mogu prikazati konačnim brojem binarnih frakcija:

$$13.3 = 13 + 0.3$$



Treba uočiti da se konačni decimalni razlomak prikazuje kao beskonačni periodički binarni razlomak.

Binarni broj se množi s potencijama baze 2 tako da se binarna točka pomakne odgovarajući broj mjesta desno ili lijevo, zavisno da li je predznak potencije pozitivan ili negativan.

Na primjer,

$$1.11 * 2^2 = 111$$

9.1.2 Oktalni brojevni sustav

Baza sustava je **B=8** a znamenke su **0,1,2,3,4,5,6,7**.

Koristi se za skraćeno zapisivanje binarnih sadržaja kada je to spretno.

Primjer:

36-bitni broj	001	110	000	101	111	001	010	011	111	000	100	001
oktalni ekviv.	1	6	0	5	7	1	2	3	7	0	4	1

9.1.3 Heksadecimalni brojevni sustav

Baza sustava je **B = 16**, a znamenke su **0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F**.

Koristi se za skraćeno zapisivanje binarnog sadržaja.

Primjer:

16-bitni broj	0111	1011	0011	1110
heksadecimalni ekviv.	7	B	3	E

9.1.4 Prikaz realnih brojeva u računalu

Standardna točnost: 32 bita (4 byte)

Dvostruka točnost: 64 bita (8 byte)

Realni brojevi standardne točnosti

Deklaracija u programskom jeziku C:

float

IEEE (Institute of Electrical and Electronics Engineers) standard 754 za prikaz realnih brojeva u standardnoj točnosti:



P predznak (P=1 negativan, P=0 pozitivan)

Karakteristika binarni eksponent + 127 (da se izbjegne prikaz negativnog eksponenta)

Mantisa normalizirana (samo jedan bit ispred binarne točke).

Primjer: Prikazati dekadski broj 5.75

$$5.75_{10} = 101.11_2 * 2^0 = 1.0111_2 * 2^2$$

Kako se normalizacijom svakog binarnog broja (osim nule) postiže oblik 1.xxxxx, vodeća jedinica ne pohranjuje se u računalu i naziva se **skrivenim bitom**.

Time se štedi jedan bit što povećava točnost.

Predznak = 0 (pozitivan broj)

Binarni eksponent = 2

Mantisa (cijela)

Mantisa (bez skrivenog bita) 0111

→ **K = 2 + 127 = 129 = (1000 0001)₂**

1.0111

Rezultat: 0 10000001 011100000000000000000000

ili 0100 0000 1011 1000 0000 0000 0000 0000

4 0 B 8 0 0 0 0 (heksadecimalno)

Primjeri:

$$2 = 10_2 * 2^0 = 1_2 * 2^1 = 0100\ 0000\ 0000\ 0000 \dots 0000\ 0000 = 4000\ 0000\ \text{hex}$$

$$P = 0, K = 1 + 127 = 128\ (10000000), M = (1.)\ 000\ 0000 \dots 0000\ 0000$$

$$-2 = -10_2 * 2^0 = -1_2 * 2^1 = 1100\ 0000\ 0000\ 0000 \dots 0000\ 0000 = C000\ 0000\ \text{hex}$$

Jednako kao 2, ali P = 1

$$-3.4 \cdot 10^{38} \quad -1.17 \cdot 10^{-38} \quad 0 \quad 1.17 \cdot 10^{-38} \quad 3.4 \cdot 10^{38}$$

9.1.4.1 Numeričke pogreške

Nemogućnost korištenja svih bita kod računanja

Primjer:

$$0.0001_{10} + 0.9900_{10}$$

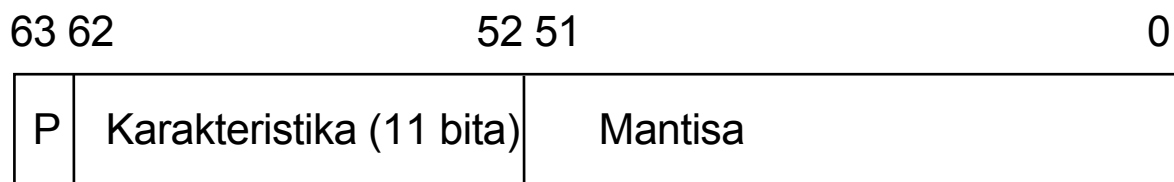
$$\begin{aligned} 0.0001_{10} : & \quad (1.)10100011011011100010111_2 \cdot 2^{-14} \\ 0.9900_{10} : & \quad (1.)11111010111000010100011_2 \cdot 2^{-1} \end{aligned}$$

Kod zbrajanja, binarne točke moraju biti poravnate:

$$\begin{array}{r} .000000000000011010001101 \cdot 2^0 \quad \text{Samo 11 od 24 bita!} \\ + .111111010111000010100011 \cdot 2^0 \\ \hline .111111010111011100110000 \cdot 2^0 = 0,9900999069214_{10} \end{array}$$

Realni brojevi dvostruke točnosti

Deklaracija u programskom jeziku C:
double



P	predznak (P=1 negativan, P=0 pozitivan)
Karakteristika	binarni eksponent + 1023 (11 bita)
Mantisa	normalizirana (52+1 bit).

Raspon:

$K \in [0, 2047]$.
 $K = 0$ rezervirana je za prikaz nule
 $K = 2047$ rezervirana je za prikaz $\square$
 $BE = K - 1023$
 $BE \in [-1022, 1023]$

Najmanji pozitivni broj različit od nule koji se može prikazati je:

$$1.0_2 * 2^{-1022} = 2.225073858507 * 10^{-308}$$

a najveći je:

$$1.1111.....111111_2 * 2^{1023} \approx 2^{1024} = 1.797693134862316 * 10^{308}$$

Točnost: 53 binarne znamenke

$$2^{53} \approx 10^x \rightarrow 53 \log 2 \approx x \log 10 \rightarrow x \approx 53 \log 2 = 15.95458977019$$

tj. približno 16 prvih važećih točnih znamenki.

Postoji još i:

long double 80 bita

Karakteristika: 15 bita

Binarni eksponent: Karakteristika - 16383

Realne konstante

1.	2.34	9e-8	8.345e+25
2f	2.34F	-1.34e5f	
1.L	2.34L	-2.5e-37L	

double
float
long double

9.2 Temeljni pojmovi programiranja

- (1) Tipovi i strukture podataka koje pseudojezik podržava:
 - statički skalarni tipovi,
 - statički strukturirani tipovi,
 - dinamički tipovi sa promjenljivom veličinom,
 - dinamički tipovi sa promjenljivom strukturom.
- (2) Osnovne kontrolne strukture koje se primjenjuju u pseudojeziku:
 - sekvenca,
 - selekcije,
 - iteracije,
 - skokovi.
- (3) Operacije ulaza i izlaza podataka:
 - ulaz/izlaz podataka za ulazno-izlazne uređaje i datoteke,
 - konverzija tekstualnog i binarnog formata podataka.
- (4) Tehnike modularizacije programa:
 - nezavisne funkcije i procedure,
 - interne funkcije i procedure,
 - rekurzivne funkcije i procedure.

9.3 Sintaksa programskih jezika

Sintaksa	- formalna ispravnost jezičnog izraza
Semantika	- s m i s a o jezičnog izraza (rečenice)
Gramatika	- skup pravila za primjenu jezika

1. BNF notacija (1960. god., J. W. Backus i P. Naur)
2. EBNF (Extended Backus-Naur form)
3. Sintaksna notacija sa zagradama (1965. god., Cobol)

a) Ekskluzivna disjunkcija

$\left\{ \begin{array}{l} \text{alfa} \\ \text{beta} \\ \text{gama} \end{array} \right\}$

b) Opcijski elementi

$\left[\begin{array}{l} \text{alfa} \end{array} \right]$

beta
_gama _

c) Metalingvističke varijable – malim slovima

broj_dijela

d) Ključne (rezervirane) riječi – velikim slovima podvučeno

WHILE logicki_izraz DO niz_naredbi END WHILE

e) Fakultativne riječi jezika – velikim slovima nepodvučeno

ADD { naziv_podatka
konstanta } ... TO { naziv_rezultata [ROUNDED] ... }

[; ON SIZE ERROR izvršena_instrukcija].

9.4 Rječnik pojmova programskog jezika C++

Apstraktna klasa je klasa koja se koristi isključivo kao osnovna klasa za nasljeđivanje, odnosno, iz takve klase nije moguće definirati objekt. Apstraktna klasa sadrži funkcijske članove koji nisu definirani (tj. sadrže samo zaglavlje) unutar te osnovne klase (tzv. čiste virtualne članove), nego u klasama koje je nasljeđuju (vidi : *čisti virtualni funkcijski član*).

Argument funkcije (parametar) je vrijednost koju proslijeđujemo funkciji i koju ona koristi u obradi. Razlikujemo stvarne i formalne argumente. Stvarni argument navodi se u pozivu funkcije, dok je formalni naveden u zaglavlju funkcije. Pri pozivu funkcije formalni argument poprima vrijednost odgovarajućeg stvarnog argumenta.

Čisti virtualni funkcijski član je funkcijski član klase koji u deklaraciji ima oznaku `=0`. Klasa je apstraktna ako sadrži barem jedan čisti virtualni funkcijski član (vidi : *apstraktna klasa*). Namjena čistog virtualnog funkcijskog člana je samo stvaranje javnog sučelja klase, dok se implementacija nalazi u izvedenim klasama.

Datoteka zaglavlja sadrži deklaracije funkcija koje tako postaju dostupne iz raznih programskih modula, pomoću pretprocesorske naredbe *#include*. Osnovna namjena im je omogućiti modularno programiranje u C / C++.

Deklaracija funkcije sadrži sve bitne informacije o funkciji koje su potrebne da bi se funkciju moglo pozivati iz drugih dijelova programa (tip i naziv funkcije, te tipovi i nazivi ulaznih argumenata). Sam programski kod nalazi se unutar definicije funkcije (vidi : *Definicija funkcije*). Deklaracija i definicija funkcije mogu biti zajedno ili razdvojene unutar različitih dijelova programskog koda.

Definicija funkcije je programski kod funkcije, koji određuje što funkcija radi, za razliku od deklaracije funkcije (vidi : *deklaracija funkcije*) koja određuje sučelje funkcije prema ostalim dijelovima programa.

Destruktor je funkcija koja oslobađa sve resurse koji su dodijeljeni objektu. Ta funkcija ima naziv koji se sastoji od znaka `~` (tilda) i naziva klase. Destruktor nema povratni tip, nema argumente i za svaku klasu može postojati samo jedan.

Dinamička alokacija memorije omogućuje dodjelu memorijskog prostora za varijablu ili objekt u toku izvođenja programa. Provodi se pomoću naredbe *new* (dealokacija s *delete*) ili pomoću funkcije *malloc* (dealokacija s *free*) koja alocira memorijski prostor zadane veličine.

Dominacija je princip prema kojem lokalni simboli prekrivaju globalne unutar svog područja (vidi : *Područje*). Primjenjuje se kod funkcija gdje su u slučaju istoimenih globalnih i lokalnih varijabli dostupne isključivo lokalne varijable. Također, princip dominacije primjenjuje se u slučajevima kada u izvedenim klasama postoje istoimeni članovi, kao i u osnovnim klasama. U tom slučaju dostupni su članovi izvedenih klasa (vidi : *Polimorfizam*).

Enkapsulacija je princip prema kojem se spajaju podaci i operacije nad tim podacima pod zajedničkim nazivom tj. unutar iste deklaracije. Primjenjuje se kod klasa, tako da one uključuju podatkovne i funkcijske članove (vidi : *Klasa*).

Funkcija je potprogram u C++, odnosno izdvojeni blok instrukcija koje predstavljaju logičku cjelinu unutar programa. Funkcija obavezno sadrži deklaraciju, koja se sastoji od tipa, naziva i liste vrijednosti koje se proslijeđuju funkciji – argumenata, te definicije, koja se sastoji od opisa podataka (tada govorimo o lokalnim podacima, jer su dostupni isključivo unutar funkcije gdje su definirani) i tijela funkcije koje sadrži naredbe za obradu. Osnovni zadatak

funkcije je da za listu ulaznih argumenata, koji se navode u njenom pozivu (tzv. stvarni argumenti) izračuna vrijednost koja je istog tipa kao i sama funkcija, odnosno, za funkciju bez tipa (ima oznaku *void* umjesto tipa podataka) da izvrši zadani blok instrukcija koji čini logičku cjelinu unutar programa.

Funkcijski član (naziva se još i **metoda**) definira skup operacija koje se mogu obaviti nad objektom, te je sastavni dio klase (uz podatkovne članove). Po svojoj strukturi funkcijski član odgovara "običnim" funkcijama, ali je lokalna za klasu unutar koje je definiran, odnosno (eventualno) u klasama koje nasljeđuju klasu u kojoj je definiran.

Globalno područje čine svi dijelovi programskog koda, odnosno sve globalno definirane varijable, objekti, funkcije, tipovi klase i pobrojenja pripadaju globalnom području. To znači da se unutar globalnog područja ne mogu nalaziti npr. dvije istoimene varijable.

Imenik sadrži deklaracije klase, funkcija i varijabli čime se one mogu isključiti iz globalnog područja imena. Namjena mu je rješavanje problema istoimenih identifikatora kod programa, na način da se u jednom trenutku učine aktivnim identifikatori iz zadanog imenika. To se može pomoću operatora dodjele područja (dvije dvotočke, ::) ili pomoću naredbe *using*, dok se imenik definira pomoću naredbe *namespace*.

Izvedena klasa je klasa koja nasljeđuje neku osnovnu klasu. Objekti izvedene klase sadrže sve funkcijske i podatkovne članove osnovne klase, ali mogu sadržavati i nova svojstva. Podrazumijevani se privatno nasljeđivanje (tj. svi članovi iz osnovne klase dostupni su samo unutar izvedene klase), ali su moguća i druga dva tipa nasljeđivanja, tj. mogu se koristiti specifikatori *public* i *protected*. Time se postiže ponovno korištenje programskog koda, odnosno, u novoj klasi je potrebno navesti samo onaj dio koda koji se razlikuje od osnovne klase.

Javno sučelje klase sastoji se od onih podatkovnih i funkcijskih članova koji imaju specifikator pristupa *public*. To znači da su dostupni i izvan klase u kojoj su definirani, pa i izvan klase koje nasljeđuju klasu u kojoj su definirani.

Kasno povezivanje predstavlja određivanje adrese poziva funkcijskog člana u toku izvođenja programa, umjesto za vrijeme prevođenja. Stvarna adresa tada se nalazi u tablici virtualnih članova, a funkcijski članovi koji se tako pozivaju moraju biti označeni ključnom riječi *virtual* (vidi : *Virtualni funkcijski član*). Kasno povezivanje dolazi do izražaja kada više objekata u lancu nasljeđivanja dijeli isti memorijski prostor. U tom slučaju se virtualni funkcijski članovi nekog objekta iz klase koja nasljeđuje osnovnu pozivaju iz te naslijeđene klase, dok se nevirtualni funkcijski članovi pozivaju iz osnovne klase.

Klasa je osnovni mehanizam objektno orijentiranog programiranja koji C++ uvodi u odnosu na C. U osnovi klasa predstavlja strukturu (koja sadrži samo podatkovne članove) proširenu potprogramima, koje nazivamo metodama ili funkcijskim članovima . U sintaksi programskog jezika C++ klasa predstavlja tip podataka, dok implementacije klase nazivamo objektima.

Klasa prijatelj je klasa koja ima mogućnost pristupa funkcijskim i podatkovnim članovima druge klase koji su deklarirani kao *private* ili *protected*. Da bi to bilo moguće potrebno je u definiciji klase navesti ključnu riječ *friend* i naziv klase koja ima pravo pristupati članovima označenim kao *private* ili *protected*. Također, pristup *klasi prijatelju* može se omogućiti samo za pojedine članove klase, tako da se oni označe kao *friend*, zajedno s nazivom klase koja ima pravo pristupa.

Konstruktor je funkcija koja inicijalizira objekt, odnosno, pridružuje početne vrijednosti podatkovnih članova objekata. Konstruktor se definira kao funkcijski član s istim imenom kao i klasa kojoj pripada, te nema specificiran povratni tip. Poziv konstruktora je automatski,

prilikom stvaranja objekta klase, a moguće je i njegovo preopterećenje, a time i različiti načini inicijalizacije objekta.

Lokalna klasa je klasa deklarirana unutar funkcije ili funkcijskog člana neke druge klase. Takva klasa dostupna je samo unutar bloka u kojem je definirana, te u blokovima unutar tog bloka. Postoje neka ograničenja kod lokalnih klasa : svi funkcijski članovi moraju biti umetnuti i ne mogu se deklarirati statički članovi.

Lokalna varijabla je varijabla koja je dostupna isključivo unutar područja gdje je definirana, tu se u prvom redu misli na funkcije, ali vrijedi također i za strukture, klase i funkcijske članove klase. Prednost korištenja lokalnih varijabli je ta da se u različitim lokalnim područjima mogu koristiti istoimeni identifikatori. Lokalna varijabla unutar područja gdje je dostupna zaklanja istoimenu globalnu varijablu, ako takva postoji.

Makro funkcija je simbol stvoren pomoću naredbe *#define*, koji može prihvaćati argumente kao i funkcija, čime se nizu naredbi dodjeljuje identifikator..

Manipulator je naredba koja se umeće u tok i služi za kontrolu formata, npr. kod ispisa sa *cout*. Manipulatori mogu biti sa i bez argumenata, a njima se određuje širina polja za ispis/učitavanje, znak za popunjavanje praznina, broj znamenki kod ispisa, brojevni sustav itd.

Međuspremnik (engl. *buffer*) je memorijski blok koji sadrži podatke koji se kasnije (tj. kad se međuspremnik napuni) upisuju na vanjske medije (npr. disk) ili su s njih pročitani (opet kao cijeli memorijski blokovi). Time se postiže veća efikasnost rada s vanjskim jedinicama. Pojam međuspremnika usko je vezan uz pojam ulaznih i izlaznih tokova (vidi : *Tok*).

Naslijeđivanje je postupak kojim se iz postojeće klase izvodi nova, tako da ta nova klasa zadržava sve funkcijske i podatkovne članove osnovne klase, ali se njihova implementacija može promijeniti (vidi : *polimorfizam*). Također nova klasa može uključivati i druge članove, osim naslijeđenih. Također, moguće je da jedna klasa naslijedi više osnovnih klasa. Pristup članovima osnovne klase definiran je odgovarajućim specifikatorom tipa naslijeđivanja (*private*, *protected* ili *public*).

Objekt predstavlja implementaciju klase, odnosno jedna klasa može imati više implementacija, objekata. Objekt se sastoji od strukture podataka (atributa, podatkovnih članova) i od postupaka koji definiraju njegovo ponašanje (metoda, funkcijskih članova).

Operator je simbol ili niz simbola kojima se pokreće izvršenje pojedine operacije. U C++ je moguće preopterećenje operatora (samo za korisnički definirane klase), tj. postojećim operatorima mogu se pridružiti nove operacije, tako da mogu raditi s drugim tipovima podataka, osim onih za koje su prvobitno namijenjeni (vidi : *operatorska funkcija*).

Operatorska funkcija definira preopterećeni operator za korisnički definirane klase. Može biti definirana kao funkcijski član klase ili izvan klase.

Pobrojenje predstavlja korisnički definirani tip podataka, takav da su moguće vrijednosti varijabli elementi pojmovnog skupa, a ne npr. brojevi ili znakovni nizovi. Svakom identifikatoru iz pojmovnog skupa pridružena je odgovarajuća numerička vrijednost.

Podrazumijevani argument predstavlja ponuđenu vrijednost formalnog argumenta čime se postiže da kod poziva funkcije nije neophodno navoditi odgovarajući stvarni argument. Time se skraćuje lista stvarnih parametara kod poziva funkcije za neke opće slučajeve (tj. one gdje se koristi podrazumijevani argument).

Područje (engl. *scope*) predstavlja dijelove programa unutar kojih su dostupni pojedini identifikatori, odnosno gdje se može pristupiti odgovarajućim varijablama, objektima, funkcijama, tipovima podataka, klasama i pobrojenjima. Razlikujemo *globalno područje* (vidi : *Globalno područje*) te lokalna područja pridružena funkcijama (tada govorimo o argumentima i lokalnim varijablama), strukturama i klasama. Razdvajanjem područja omogućeno je korištenje istoimenih identifikatora unutar različitih dijelova programa.

Pokazivač je varijabla koja sadrži adresu neke varijable ili objekta u memoriji. Takvu varijablu objekt nazivamo dinamičkim. Pokazivač se inicijalizira pomoću naredbe *new*, čime se rezervira memorijski prostor za novu dinamičku varijablu ili objekt, a deinicijalizira se s naredbom *delete*. Pokazivači omogućuju implementaciju dinamičkih struktura podataka, kao što je linearna lista ili binarno stablo.

Pokazivač datoteke je cijeli broj koji predstavlja poziciju unutar datoteke na kojoj se može izvršiti ulazna ili izlazna operacija (čitanje ili upis podataka). Pozicija je izražena u broju bajtova od početka datoteke. Operacija upisa ili čitanja podataka u/iz datoteke (npr. pomoću funkcijskog člana *write*, odnosno *read*) upisuje/učitava podatke na/sa pozicije zadane datotečnim pokazivačem, a zatim pomiče pokazivač na poziciju slijedećeg zapisa. Klasa *fstream* definira osnovne operacije nad datotečnim pokazivačem : postavljanje vrijednosti pokazivača (operacija *seek*) i očitavanje vrijednosti pokazivača (operacija *tell*).

Pokazivač *this* je pokazivač na objekt čiji je funkcijski član pozvan. Prilikom poziva svakog funkcijskog člana klase proslijeđuje se pokazivač na objekt kojemu taj član pripada, a tome se može pristupiti pomoću ključne riječi *this*.

Polimorfizam predstavlja mogućnost promatranja objekta kroz različite klase u stablu nasljeđivanja. Time se omogućuje definiranje operacija koje ne ovise o tipu, odnosno objekti izvedenih klasa zadržavaju sve funkcijske i podatkovne članove osnovnih klasa, ali im se može pridružiti drukčija implementacija i nova svojstva (vidi : *Dominacija*).

Polje (polje podataka) je niz konačnog broja istovrsnih podataka, elemenata polja. Svi elementi polja imaju isti naziv i tip, a pojedinačno se dohvaćaju pomoću brojčanog indeksa, koji se navodi unutar uglatih zagrada. Tipovi podataka za elemente polja mogu biti ugrađeni ili korisnički definirani (npr. *struktura*). Moguća su jednodimenzionalna i višedimenzionalna polja, pri čemu se za adresiranje koristi toliko indeksa kolika je dimenzija polja.

Predložak funkcije omogućuje korištenje istog koda funkcije za različite tipove argumenata. To znači da takva funkcija koristi argumente onih tipova kojih su stvarni parametri. Predložak funkcije se definira pomoću ključne riječi *template*.

Predložak klase slično predlošku funkcije definira opću klasu koja se zatim konkretizira stvarnim tipovima. Stvarni tipovi podataka (koji se mogu razlikovati kod implementacije) zamjenjuju se općim tipom, čime se izbjegava višestruko pisanje koda.

Preopterećenje funkcije predstavlja korištenje istog imena za različite inačice funkcija. U tom slučaju inačice se razlikuju po listi argumenata, odnosno, odgovarajuća inačica biti će pozvana u slučaju da je kod poziva funkcije navedena odgovarajuća lista stvarnih argumenata. Preopterećenjem se smisleno poistovjećuju funkcije koje obavljaju slične radnje nad različitim tipovima podataka.

Preopterećeni operator je operator koji osim predefiniranih operacija specificira i korisnički definirane. U većini slučajeva su to istovrsne operacije predefiniranim, ali se vrše nad podacima drugih tipova, pa je potrebno izvršiti konverziju.

Pretprocesorska naredba predstavlja uputu programu prevoditelju koja mu govori o načinu prevođenja programa. Ispred takve naredbe nalazi se znak `#`. Različite pretprocesorske naredbe koriste se najviše za uključivanje vanjskih dijelova programa (naredba `#include`), za definiranje vrijednosti simbola (`#define`), za umetanje preddefiniranih makro imena, i za uvjetno prevođenje.

Referenca je posebna varijanta pokazivača, a predstavlja adresu neke varijable ili objekta. Definira se tako da se ispred naziva varijable postavi znak `&` (adresni operator), odnosno, `&` nam govori da se ne radi o sadržaju neke varijable nego o njenoj adresi. Referenca se može koristiti kao drugo ime za varijablu ili kao parametar funkcije, u kom slučaju se omogućuje promjena vrijednosti stvarnih parametara iz funkcije.

Rekurzivna funkcija je funkcija koja poziva samu sebe, tj. može se pozivati i iz tijela same te funkcije, a ne samo izvana. To svojstvo moguće je zato jer se svakim pozivom funkcije stvaraju nove lokalne varijable, koje su potpuno neovisne o varijablama koje su bile dostupne kod poziva. Rekurzivnim funkcijama mogu se rješavati rekurzivne matematičke jednadžbe odnosno implementirati rekurzivni algoritmi.

Skrivanje podataka je princip prema kojem podaci koji su privatni za neki objekt ne smiju biti dostupni drugim dijelovima programa, a postiže se odgovarajućim specifikatorima pristupa (vidi : *Specifikatori pristupa*).

Standardna biblioteka sadrži funkcije, klase i predloške koji se često koriste u pisanju programa, te time oslobađa programera njihovog pisanja. Standardna biblioteka jezika C++ sadrži još i definicije makro funkcija i makro imena, vrijednosti simboličkih konstanti, tipove podataka i objekte. Deklaracije i definicije elemenata standardne C++ biblioteke raspodijeljene su kroz 32 datoteke zaglavlja (vidi : *Datoteke zaglavlja*), a osim njih uključene su i datoteke zaglavlja jezika C.

Specifikatori pristupa određuju dostupnost pojedinih funkcijskih i podatkovnih članova klase. Razlikujemo slijedeće specifikatore pristupa :

- *public* : određuje da su pripadni članovi klase dostupni svim funkcijama u programu,
- *private* : pripadnim članovima klase mogu pristupiti isključivo funkcijski članovi te iste klase (izuzetak je *klasa prijatelj*) i
- *protected* : pripadnim članovima klase mogu pristupiti funkcijski članovi iste klase i klase koje su iz te klase izvedene.

Struktura je složeni tip podataka, koji se sastoji od podataka različitih tipova i naziva. Struktura se definira pomoću ključne riječi *struct*, a namjena joj je grupiranje više različitih podataka koji čine logičku cjelinu u jedan zajednički podatak, radi lakše manipulacije, npr. zapisivanja u datoteku. Za razliku od klase (vidi : *Klasa*) struktura sadrži samo podatkovne članove.

Tok je klasa koja definira ulazne i izlazne operacije s vanjskim jedinicama. Različiti tokovi definirani su unutar standardnih biblioteka jezika C++. Podaci koji se čitaju/upisuju nalaze se u međuspremniku (vidi : *međuspremnik*).

Ugniježdjena klasa je klasa koja je definirana u području neke druge klase. Takva klasa naziva se još i lokalna klasa.

Umetnuti članovi su oni podatkovni i funkcijski članovi klase koji se nalaze unutar istog programskog bloka s deklaracijom klase, tj. ključna riječ *inline* se podrazumijeva, pa je nije potrebno navoditi (vidi : *Umetnuta funkcija*).

Umetnuta funkcija je funkcija koja u deklaraciji sadrži ključnu riječ *inline*, kojom određujemo da se svaki poziv funkcije zamjenjuje odgovarajućim programskim kodom. Korištenjem umetnutih funkcija izbjegava se utrošak vremena na poziv funkcije i povratak iz nje, što ubrzava izvođenje programa, ali se zato povećava duljina njegovog izvedbenog koda.

Unija sadrži podatkovne i funkcijske članove, slično strukturi, odnosno klasi. Razlika je u tome što svi podatkovni članovi unije dijele isti memorijski prostor, što znači da se prilikom upisa vrijednosti jednog podatkovnog člana automatski mijenjaju vrijednosti svih ostalih.

Vanjsko povezivanje indikatora sugerira prevoditelju da je odgovarajući modul definiran u drugom programskom modulu. Indikatori se učine dostupnima i iz druge datoteke pomoću ključne riječi *extern*.

Virtualni funkcijski član je funkcijski član čija adresa se nalazi u tablici virtualnih funkcijskih članova, odnosno, ne dodjeljuje se kod prevođenja programa nego tek u toku izvođenja (vidi : *Kasno povezivanje*).

Zastavica označava odgovarajući način formatiranja ispisa unutar klase *ios*. Pomoću zastavica moguća je precizna kontrola ispisa podataka (način poravnanja, brojevni sustav, notacija itd.). Za manipuliranje zastavicama definirani su potrebni funkcijski članovi klase *ios*.